

Abductive Explanations for Groups of Similar Samples

Anonymous submission

Abstract

A formal abductive explanation (AXp) is a subset of features of a sample such that any sample fixing them receives the same classification. An AXp fixes the values of a single sample, so a measurement error, for example, can invalidate it. The sets of AXps for similar samples overlap only partially, so computing a single random AXp for each may produce very different AXps, while computing all of them may result in a large set from which it is difficult to choose. We address these challenges by introducing group AXps, explanations that hold for all samples in a group. We introduce algorithms for clustering a large set of samples into groups sharing a group AXp as well as constructing a maximal group around an input sample. An empirical evaluation shows that our algorithms cluster a set of samples into groups of usually non-trivial size within a reasonable time and, for an input sample, produce fewer and more general explanations.

Introduction

A growing number of real-world decisions rely on machine learning models (Karimi et al. 2023). Research into explaining these models has blossomed in response (Confalonieri et al. 2021; Minh et al. 2022; Bodria et al. 2023; Marques-Silva 2024), but many of the proposed methods can produce misleading explanations (Kumar et al. 2020; Huang and Marques-Silva 2024; Marques-Silva and Huang 2024). Popular approaches such as SHAP (Lundberg and Lee 2017) and LIME (Ribeiro, Singh, and Guestrin 2016) are heuristic and therefore offer no formal guarantees. Formal abductive explanations (AXps) (Shih, Choi, and Darwiche 2018; Ignatiev, Narodytska, and Marques-Silva 2019; Marques-Silva and Ignatiev 2022) answer with a proof instead. An AXp is a subset of the features whose values alone guarantee that the classifier assigns the same label to any input that matches those values.

Consider the following example. A clinic deploys a neural network that flags patients at risk, and regulation entitles every patient to “meaningful information about the logic involved”¹ in the decision (Karimi et al. 2023). For each flagged

patient, instead of the approximate importance scores of a heuristic method, an AXp would identify the measurements, say blood pressure and age, that account for the flag.

But AXps are fragile to input variation, they are not unique, and they only apply to individual samples. An AXp that relies on a blood pressure of 140.0 mmHg could fail for a reading of 140.1 mmHg, so a measurement error voids the guarantee. The literature recognizes the need for explanations that are robust to (small) perturbations (Lakkaraju, Arsov, and Bastani 2020; Fokkema, de Heide, and van Erven 2023). Each AXp is also one of dozens (Audemard et al. 2022), and it is not clear which one the user should see (Liffiton et al. 2016; Yu et al. 2024). Lastly, distinct similar samples might receive very different explanations, such as five of the clinic’s patients receiving five different AXps for the same flag. These three limitations share a root: an AXp fixes the exact values of its relevant features. It explains a point when the user needs a region.

An existing approach to tackle these limitations replaces the point with a bounding box. Inflated AXps (Izza et al. 2024b, 2025), for example, maximize the bounds of the relevant features, so that the classifier assigns the same label to every instance inside the resulting box. But maximality lets some features take the room of others, and the greedy algorithm that inflates the bounds, one feature at a time, often leaves the last features with no room to vary at all. In this paper, we derive the bounds from data instead. We begin by formalizing δ -robust AXps, which take a vector of bounds δ as input and hold while each relevant feature i ranges over $[\delta_i^-, \delta_i^+]$, capturing both standard AXps, whose bounds are all the feature values, and inflated AXps, whose bounds are all maximal. The user can supply these bounds directly whenever the application fixes them, as with the known measurement error of an instrument.

Our main contribution, the semi-local explanations *group AXps*, builds on δ -robust AXps. A group AXp is a δ -robust AXp whose bounds span a set of samples. The difficulty then moves from growing the bounds to finding the right groups. We tackle the challenge from two directions, both assuming that we have multiple samples sharing the same classification. First, to *explain many samples at once*, we *partition them into groups with at least one group AXp*, and we call such a partition a *cover*. Our five similar patients, for example, would land in the same group and receive only the

¹<https://gdpr-text.com/read/article-13/>

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

explanations that hold for all of them. Second, to explain a *single sample robustly* and to *trade its many AXps for a few that generalize*, we grow a *maximal group* around it, so that the sample and its neighbours share at least one explanation and the accident of an exact reading no longer decides what the clinician sees.

Concretely, our contributions are as follows.

- We investigate δ -robust AXps and provide an algorithm that computes one at a single oracle call more than a standard AXp.
- We introduce group AXps together with greedy algorithms for two use cases, covering many samples at once and growing a maximal group around a sample.
- We evaluate both use cases on five benchmark datasets. On average, a maximal group reduces the number of explanations to almost one and contains up to 12 samples on average for a class, and a notable share of the cover groups holds more than 5 samples. Finally, constructing a maximal group around a sample and computing a group AXp is significantly faster than inflating an explanation given a sample.

Related Work

Feature selection approaches divide into *abductive* explanations (Shih, Choi, and Darwiche 2018; Ignatiev, Narodytska, and Marques-Silva 2019; Izza and Marques-Silva 2021; Ignatiev and Marques-Silva 2021; Wu, Wu, and Barrett 2023; Audemard et al. 2023), which answer *why this classification*, and *counterfactual* explanations (Wachter, Mittelstadt, and Russell 2018; Karimi et al. 2020; Parmentier and Vidal 2021; Karimi et al. 2023; Guidotti 2024), which answer *how to obtain a different classification*. Robustness has a longer history for counterfactual explanations (Jiang et al. 2023, 2024) than for AXps. For AXps, robustness appears as distance-restricted AXps (Izza et al. 2024a; Huang and Marques-Silva 2023) and ϵ -robust explanations (Wu, Wu, and Barrett 2023), where the *relevant* features, those in the AXp, stay fixed while the *irrelevant* features, those outside of the AXp, vary by up to ϵ . In contrast, δ -robust AXp allow each relevant feature to vary inside an interval of its own, regardless of how irrelevant features vary. Distance-restricted explanations are a relaxation of AXps while δ -robust AXps are a stronger condition. In the parlance of Jiang et al. (2024), δ -robust AXps are robust against noisy execution. Other refinements to AXp include probabilistic AXps, which relax the sufficiency condition to a given probability of the classification remaining the same, in order to limit the size of explanations (Wäldchen et al. 2021).

The closest counterparts of δ -robust AXps are inflated AXps (Izza et al. 2024b), which maximize the bound of each relevant feature. Space explanations (Labbaïf et al. 2025) also find explanations for a neural network over continuous regions of the feature space, but they characterize maximal regions via interpolated formulas without feature-minimality.

A separate line of work confronts the multiplicity of AXps. MARCO-style algorithms enumerate the collection (Liffiton et al. 2016; Ignatiev and Marques-Silva 2021), and formal feature attribution summarizes it, scoring each feature by the

share of AXps that rely on it (Yu et al. 2024). Our maximal groups generalize instead, trading the collection for a few new explanations that provably hold across a whole neighbourhood of samples that the model classifies alike.

Group explanations pose the challenge of covering multiple samples with a single explanation. Recent work studies them for counterfactual explanations (Warren et al. 2023; Carrizosa, Ramírez-Ayerbe, and Morales 2024b,a; Wielopolski et al. 2024), but, to the best of our knowledge, no prior work considers them for AXps. Anchors (Ribeiro, Singh, and Guestrin 2018) come closest among the heuristic methods, since a rule that holds around one sample also covers the neighbours that satisfy it, but they estimate the rule’s precision by sampling, while every explanation in this paper carries a proof.

Abductive Explanations

Let $\mathbb{F} = \{1, \dots, d\}$ be a set of features. A sample is a vector $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$, where x_i denotes the value of $\mathbf{x}$ for feature $i \in \mathbb{F}$. A classifier is a function $f : \mathbb{R}^d \rightarrow \{1, \dots, k\}$ mapping each sample $\mathbf{x}$ to a class $c = f(\mathbf{x})$.

Definition 1. A subset of features $E \subseteq \mathbb{F}$ is an *abductive explanation (AXp)* for $f(\mathbf{x})$ if

$$\forall \mathbf{z} \in \mathbb{R}^d : \bigwedge_{i \in E} (z_i = x_i) \rightarrow f(\mathbf{z}) = f(\mathbf{x})$$

and there is no proper subset of E satisfying this condition.

We call the features in E *relevant* and those in $\mathbb{F} \setminus E$ *irrelevant*.

δ -Robust AXps

AXps do not extend to samples that differ in the value of a relevant feature, however small the difference (Ignatiev, Narodytska, and Marques-Silva 2019), so a measurement error already voids them in practice. An application often knows how much variation it must tolerate, such as the measurement error of an instrument. We therefore consider δ -robust AXps, which take the bounds as input.

Definition 2. Let $E \subseteq \mathbb{F}$ be a subset of features and let $\delta = \{(\delta_i^-, \delta_i^+) \in \mathbb{R}^2 \mid i \in \mathbb{F}\}$ be a set of robustness bounds. The pair (E, δ) is a δ -robust AXp for $f(\mathbf{x})$ if

$$\forall \mathbf{z} \in \mathbb{R}^d : \bigwedge_{i \in E} (\delta_i^- \leq z_i \leq \delta_i^+) \rightarrow f(\mathbf{z}) = f(\mathbf{x}), \quad (1)$$

holds and no proper subset of E satisfies it for δ . When $\delta = \{(x_i - v, x_i + v) \mid i \in \mathbb{F}\}$, we write (E, v) and call it a v -robust AXp.

Standard AXps are then 0-robust AXps.

We compute a δ -robust AXp with constraint solving by adapting the standard deletion-based algorithm for minimal unsatisfiable subsets of constraints (Chinneck and Dravnieks 1991). We one-hot encode categorical features and normalize numerical features to $[0, 1]$. We assume that a set of logical constraints encodes the classifier, and that an oracle decides whether a set of constraints entails another. The function $\text{GUARANTEED}(\phi, \psi)$ encapsulates the encoding and the oracle

Algorithm 1 Computing a δ -robust AXp.

```

1: function COMPUTEROBUSTAXP( $f, \mathbb{F}, \mathbf{x}, \delta$ )
2:    $\psi \leftarrow \bigwedge_{i \in \mathbb{F}} (\delta_i^- \leq z_i \leq \delta_i^+)$ 
3:   if  $\neg \text{GUARANTEED}(\psi, f(\mathbf{z}) = f(\mathbf{x}))$  then
4:     return False
5:    $E \leftarrow \mathbb{F}$ 
6:   for each feature  $i$  in  $\text{TRAVERSALORDER}(\mathbb{F})$  do
7:      $\phi \leftarrow \bigwedge_{j \in E \setminus \{i\}} (\delta_j^- \leq z_j \leq \delta_j^+)$ 
8:      $\phi \leftarrow \phi \wedge \bigwedge_{j \in \mathbb{F} \setminus (E \setminus \{i\})} (0 \leq z_j \leq 1)$ 
9:     if  $\text{GUARANTEED}(\phi, f(\mathbf{z}) = f(\mathbf{x}))$  then
10:       $E \leftarrow E \setminus \{i\}$ 
11:   return  $E$ 

```

and returns `True` iff every assignment that satisfies ϕ also satisfies ψ . The function `TRAVERSALORDER` sorts the set of features $\mathbb{F}$, and any permutation works, though some orders yield preferable explanations (Wu, Wu, and Barrett 2023), and ours sorts randomly. Each call to `GUARANTEED` decides whether the network keeps a classification over a box, a problem that is NP-complete for ReLU networks (Katz et al. 2017), so a call dominates all the work around it and we count the calls of every algorithm as its cost.

Algorithm 1 first checks whether the oracle guarantees the classification throughout the box of δ (line 3), and returns `False` if it does not, since then no δ -robust AXp exists for δ . Otherwise it over-approximates E as $\mathbb{F}$ and tests the features in the order of `TRAVERSALORDER` (lines 6–10). Each test fixes the still-relevant features to their robustness intervals, frees the tested feature together with the discarded ones over the whole domain, and drops the tested feature from E if the oracle still guarantees the classification. The surviving set E makes (E, δ) a δ -robust AXp for $f(\mathbf{x})$. The initial check makes one oracle call and the loop one more per feature, so the algorithm makes exactly $d + 1$ calls, one more than a standard AXp, whose initial check merely evaluates $f(\mathbf{x})$.

Inflated AXps

An application that cannot name its bounds may ask for the widest ones the classifier allows, so Izza et al. (2024b) proposed inflated AXps.

Definition 3. A δ -robust AXp (E, δ) is an inflated AXp if δ is maximal, that is, there are no bounds γ that satisfy (1) with $\gamma_i^- \leq \delta_i^-$ and $\delta_i^+ \leq \gamma_i^+$ for every $i \in E$, and $\gamma_j^- < \delta_j^-$ or $\delta_j^+ < \gamma_j^+$ for some $j \in E$.

To inflate an AXp E , Izza et al. (2024b) give each relevant feature $i \in E$ a bound (δ_i^-, δ_i^+) that starts at (x_i, x_i) . They increase the upper bound δ_i^+ by a small fixed precision η and call an oracle to check whether f assigns c to every sample $\mathbf{z}$ within the bounds. The bound grows until the explanation breaks, they revert to the last valid value $\delta_i^+ - \eta$ and repeat the procedure for the lower bound δ_i^- , and the final bounds are maximal up to η .

This algorithm maximizes the bounds with at least $2 \cdot |E|$ oracle calls, and an extra call for every widening. It inflates the bounds one at a time, so the features it reaches early

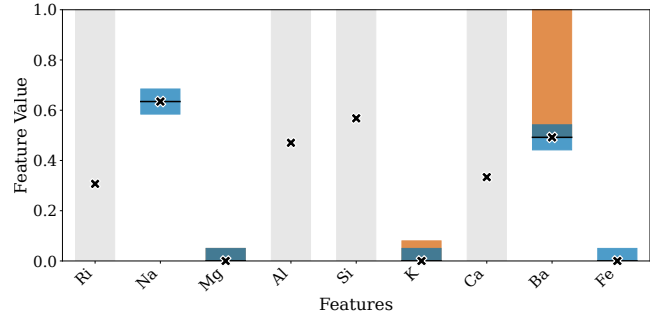


Figure 1: An AXp (in black), an inflated AXp (in orange), and a v -robust AXp with $v = 0.05$ (in blue) for a sample of the Glass dataset (in black). The irrelevant features span their whole domain (in grey).

tend to end up with much larger bounds than the last ones, which sometimes remain fixed. And its result depends on the inflation order, one of $(2 \cdot |E|)!$, so the order decides which inflated AXp the user sees. A δ -robust AXp, in contrast, needs $d + 1$ oracle calls and its bounds do not depend on an order, but it may not exist. The bounds may be too large to guarantee the classification. An inflated AXp always exists, as in the worst case the algorithm widens no bounds and returns the AXp it started with.

Example 1. Figure 1 contrasts the three kinds of AXp on a Glass sample. The standard AXp fixes each relevant feature to its value in the sample. The inflated AXp stretches ‘Ba’ over half its domain but leaves ‘Na’ and ‘Fe’ pinned without a box, while the v -robust AXp guarantees every relevant feature a width between 0.05 and 0.10, the narrower bands falling to features whose value sits at the edge of its domain.

Group AXps

Our clinic knows its bounds, since the blood-pressure monitor carries a known measurement error, but in general the right bounds are unclear. We thus propose a way to compute δ -robust AXps by looking at groups of similar samples with the same classification, introducing group AXps.

Definition 4. Let X be a set of samples sharing the same classification. A group AXp for X is a δ -robust AXp $(E, \delta(X))$ where

$$\delta(X) = \{(\min_{\mathbf{x} \in X} x_i, \max_{\mathbf{x} \in X} x_i) \mid i \in \mathbb{F}\}.$$

Every sample of X falls inside $\delta(X)$ by construction, so a single group AXp covers the whole group, that is, it is a valid explanation for each individual sample in the group. A group AXp thus identifies the features that account for the shared classification, and it is semi-local, applying neither to a single sample nor to the entire model but to a collection of samples. Group AXps work best when the samples resemble each other, so finding appropriate groups is the central challenge of our evaluation.

A group AXp derives its bounding box $\delta(X)$ from data, so it needs no special rule for categorical features. Every interval stretches between values that samples of the group

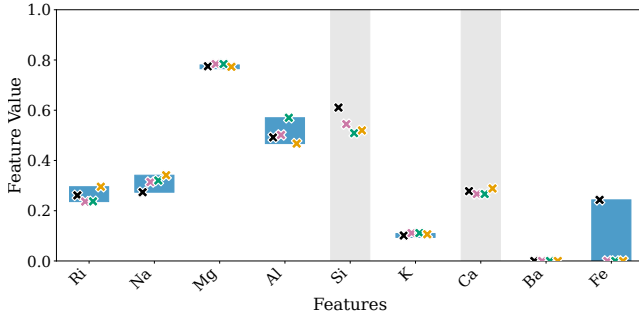


Figure 2: A group AXp (in blue) for four samples from the Glass dataset (in black, purple, green, and orange). The irrelevant features span their whole domain (in grey).

take, and a categorical feature thus never receives a bound that stops between two of its values. A binary feature, for example, stays pinned when the group agrees on it and spans both values when the group mixes them.

Example 2. Figure 2 shows a group AXp (E, δ) for four Glass samples that share a class. For each relevant feature, the box spans from the smallest to the largest value the feature takes across the four. The classifier assigns their shared class to any sample whose features all fall within these intervals, and thus to all four at once.

The function `COMPUTEGROUPAXP` builds the bounding box $\delta(X)$ of the definition and calls `COMPUTEROBUSTAXP` (Algorithm 1) with it and any sample of X as representative. A `False` from the initial check says that the group has no group AXp, and otherwise the call returns a set E such that $(E, \delta(X))$ is a group AXp for X . Building $\delta(X)$ needs no oracle calls, so a group AXp costs the same $d + 1$ calls as a δ -robust AXp.

Before describing algorithms for the two use cases of the introduction, covering a set of samples and growing a maximal group around a sample, we introduce some helpful notions. The function `HASGROUPAXP` verifies whether a group has a group AXp by performing the initial check of Algorithm 1 (line 3) for the group’s bounding box with a single oracle call. Once an application fixes its groups, the loop of Algorithm 1 (lines 6–10) yields their group AXps at d oracle calls per group.

The non-existence of a group AXp is monotonic. Every sample added to a group can only widen the intervals of δ , and a wider δ is more likely to cross a decision boundary, so larger groups fail the test more often.

Proposition 1. *Let $X \subseteq X'$ be sets of samples sharing the same classification. If X has no group AXp, then X' has no group AXp either.*

Proof. If X' has a group AXp $(E, \delta(X'))$, then each interval of $\delta(X)$ lies inside its counterpart in $\delta(X')$, so E satisfies (1) for $\delta(X)$, and discarding features from E until no more can go yields a group AXp for X . $\square$

Both use cases grow their groups greedily, and the *area* lets them rank candidate groups without additional oracle calls.

Algorithm 2 Covering X with group AXps, top-down.

Require: $f(\mathbf{x}) = f(\mathbf{x}')$ for all $\mathbf{x}, \mathbf{x}' \in X$
1: **function** `COVERTOPDOWN`($f, \mathbb{F}, X$)
2: **if** `HASGROUPAXP`($f, \mathbb{F}, X$) **then**
3: **return** $\{X\}$
4: **return** $\bigcup_{X' \in \text{SPLIT}(X)} \text{COVERTOPDOWN}(f, \mathbb{F}, X')$

Definition 5. *The area of a set of samples X is*

$$\text{area}(X) = \sum_{i \in \mathbb{F}} \left(\max_{\mathbf{x} \in X} x_i - \min_{\mathbf{x} \in X} x_i \right).$$

The area totals the edge lengths of the group’s bounding box, so a smaller area marks a tighter group. Multiplying the edge lengths instead would tie at zero every group that agrees on a single feature, and similar samples often agree on several.

Covering Many Samples with Group AXps

In our first use case, users need to explain many samples at once. The clinician of our introduction, for example, may want to know which measurements drive the flag, not for one patient but across all flagged patients. A single group AXp rarely stretches that far, as the bounding box of a large set of samples usually crosses a decision boundary, so we partition the set into groups that each have a group AXp.

Definition 6. *Let X be a set of samples sharing the same classification. A cover C of X is a partition of X into groups that each have a group AXp.*

A cover then explains all of X with one group AXp per group. Every X has a cover, since in the worst case every $\mathbf{x} \in X$ can form its own group $\{\mathbf{x}\}$, but we aim to find covers with few groups. We propose two algorithms for computing them.

Algorithm 2 covers the set of samples X top-down. It first builds a linkage tree over the class by greedy agglomeration, merging at each step the two clusters whose union has the smallest area, and then descends the tree from the root. A node whose box passes `HASGROUPAXP` becomes a group of the cover, and a node that fails splits into its two children (`SPLIT`). The recursion terminates upon a valid cover, because a singleton’s box is the sample itself, and the pool contains only samples that the model assigns to the class. Building the tree needs no oracle calls. The descent needs one call per visited node, and the tree holds $2|X| - 1$ nodes, so covering X needs at most $2|X| - 1$ calls.

Algorithm 3 builds the cover bottom-up instead. Every sample starts as its own group, and at each step the algorithm picks the unbanned pair of groups whose union has the smallest area and tries to merge it. The algorithm merges a pair that passes `HASGROUPAXP` and bans a pair that fails from every later test. By Proposition 1, a merged group inherits the bans of both its parts, since a pair that fails also fails inside any larger union. The algorithm stops once it has banned every remaining pair, so no two groups of the final cover can merge any further. Each pair faces the oracle at most once, since a failed pair stays banned and a merge retires both of

Algorithm 3 Covering X with group AXps, bottom-up.

Require: $f(\mathbf{x}) = f(\mathbf{x}')$ for all $\mathbf{x}, \mathbf{x}' \in X$
1: **function** COVERBOTTOMUP($f, \mathbb{F}, X$)
2: $C \leftarrow \{\{\mathbf{x}\} \mid \mathbf{x} \in X\}$
3: **while** some pair in C is not banned **do**
4: $\{A, B\} \leftarrow$ unbanned pair minimizing area
5: **if** HASGROUPAXP($f, \mathbb{F}, A \cup B$) **then**
6: replace A and B in C by $A \cup B$
7: **else**
8: ban $\{A, B\}$
9: **return** C

Algorithm 4 Growing a maximal group around a sample.

Require: $\mathbf{x} \in X$ and $f(\mathbf{x}') = f(\mathbf{x})$ for all $\mathbf{x}' \in X$
1: **function** MAXIMALGROUP($f, \mathbb{F}, X, \mathbf{x}$)
2: $G \leftarrow \{\mathbf{x}\}$
3: $X \leftarrow X \setminus \{\mathbf{x}\}$
4: **while** $X \neq \emptyset$ **do**
5: $\mathbf{z} \leftarrow$ sample of X minimizing area($G \cup \{\mathbf{z}\}$)
6: $X \leftarrow X \setminus \{\mathbf{z}\}$
7: **if** HASGROUPAXP($f, \mathbb{F}, G \cup \{\mathbf{z}\}$) **then**
8: $G \leftarrow G \cup \{\mathbf{z}\}$
9: **return** G

its groups, and a run creates at most $2|X| - 1$ groups, so the algorithm uses $O(|X|^2)$ oracle calls.

The two algorithms trade cover quality against oracle calls. The top-down algorithm commits to the merges of one pre-computed tree, so it settles X in at most $2|X| - 1$ calls but never recovers from a poor early split. The bottom-up algorithm reconsiders every pair after each merge, so it reaches groups that the tree never proposes and pays for the search with the calls of its failed merges. Both algorithms lean on the area, since a candidate with a tighter box is less likely to cross a decision boundary.

Explaining a Sample with a Maximal Group

Our second use case trades the many AXps of a sample, exponentially many in the worst case (Audemard et al. 2022), for explanations that *generalize*. Around the sample, we grow a maximal group of same-class samples and keep the group AXps of that group. A group AXp of the group is minimal for the group’s box rather than for the sample, so the group creates new explanations instead of choosing among the sample’s own, and each of them holds across the whole neighbourhood by definition. Every group AXp of the group is also a sufficient reason for the sample itself, and the group never leaves the sample without one, since it has a group AXp by construction.

Algorithm 4 grows the maximal group around a sample $\mathbf{x}$ inside a pool X of samples sharing its classification. Starting from $\mathbf{x}$ alone, the algorithm picks at each step the sample $\mathbf{z}$ of X that increases the group’s area the least (line 5) and tests the enlarged group with HASGROUPAXP. A candidate that passes joins the group. A candidate that fails never returns, since the group only grows and, by Proposition 1, it

	Diabetes	Glass	Leaf	Parkinsons	Wine
Samples	442	214	340	195	178
Classes	2	7	30	2	3
Features	10	9	14	22	13
Majority	0.506	0.349	0.044	0.744	0.389
Accuracy	0.719	0.628	0.750	0.923	1.000
Macro-F1	0.719	0.586	0.766	0.902	1.000

Table 1: Dataset characteristics and classifier performance. “Majority” is the fraction of the most frequent class.

would fail again with any larger group. The resulting group is therefore maximal, as no discarded sample can join it. And the algorithm tests each candidate once, so it needs $|X| - 1$ oracle calls.

Empirical Evaluation

We empirically evaluate our approaches in light of five questions. The supplementary material contains our code along with all the parameters we use, and we will release it as open source. We first focus on group AXps and address (1) how many fewer group AXps are there than AXps, (2) how long does it take to compute a cover with the different algorithms and how big are the groups, and (3) how many explanations does a maximal group around a sample leave?

We then test the applicability of group AXps as a complementary method to inflated AXps and address (4) how long does it take to find a maximal group AXp compared to finding an inflated explanation, and (5) how often does the greedy inflation of an AXp completely fix some feature value?

Datasets We use the datasets Diabetes (Efron et al. 2004), Glass (German 1987), Leaf (Silva and Marçal 2014), Parkinsons (Little 2007), and Wine (Aeberhard and Forina 1992), five tabular datasets whose features are all numerical except for two categorical features of Diabetes, ‘Sex’ and ‘Age’. Diabetes is a regression dataset, so we binarize its target at the median disease-progression score into a balanced two-class problem. Table 1 summarizes their characteristics.

Models We train for each dataset a fully-connected neural network (NN) with two hidden layers of 32 ReLU neurons on an 80%/20% train-test split. We optimize with Adam at a learning rate of 0.001 and a batch size of 16 for up to 1,000 epochs, stop early when the validation loss on a 10% split of the training data fails to improve for 10 epochs, and weight each class inversely to its frequency. We reuse the same architecture and untuned hyperparameters across all datasets to get comparable results. Table 1 summarizes the performance of the trained NNs. The accuracy stays modest on Glass, but AXps explain the model rather than the data, so a weak classifier receives explanations as faithful as a strong one. Our covering and grouping experiments use only the correctly classified samples, and they draw them from the whole dataset rather than from the test split alone. A seed of 0 fixes every split and the training, and the experiments that compute or inflate a single AXp average the five random orders of the seeds 1 to 5.

	Diabetes	Glass	Leaf	Parkinsons	Wine
Top-down cover (Algorithm 2)					
Group Size	2.2	2.3	1.6	2.2	2.9
Build time (s)	32.1	5.0	3.6	58.6	11.1
Bottom-up cover (Algorithm 3)					
Group Size	2.9	2.7	1.7	2.5	3.5
Build time (s)	77.9	10.2	4.5	140.0	27.7

Table 2: Mean group size and time to build the cover for the two algorithms.

Verifier We use α, β -CROWN (Wang et al. 2021) as the NN verifier for the GUARANTEED oracle. Our approach relies on no specific feature of it, so any sound and complete NN verifier can replace it. Each oracle call runs with the default branch-and-bound budget of α, β -CROWN, 360 seconds, and every algorithm treats an undecided call as a failure.

Hardware We run each task on a single core of a Xeon E5-2680 v3 cluster with 16 GB of memory. Tasks that compute a single AXp run with a one-hour timeout, and tasks that enumerate all AXps run with a twelve-hour timeout. The `requirements.txt` file in our code pins the exact version of every library.

Covers Explain Many Samples with a Few AXps

Our first experiment measures the size of the groups that our algorithms find. We use every correctly classified sample of one class as the initial pool to cover.

Table 2 reports the covers that Algorithms 2 and 3 produce, averaged over the classes of each dataset, and Figure 3 shows how the covers distribute the samples over group sizes. Both algorithms place most samples in a group of two or more, while the boundary samples that resist grouping stay singletons and keep the mean group size low. The bottom-up covers beat the top-down ones on every dataset in terms of group size. They form larger groups and leave fewer samples alone, raising the grouped share of Diabetes from 81 to 92% and of Wine from 92 to 96%. Finding the better covers takes longer, since the failed merges consume extra oracle calls. For example, covering a class of Parkinsons takes 140 instead of 59 seconds, and one of Diabetes 78 instead of 32. The top-down algorithm thus offers a good cover in half the time, and the bottom-up algorithm a better one when the user can afford the runtime.

A Maximal Group Replaces Many AXps with Few

Our second experiment focuses on constructing a maximal group around an input sample. We grow a maximal group around every correctly classified sample with Algorithm 4, enumerate the AXps of the sample and the group AXps of its group with MARCO (Liffiton et al. 2016; Ignatiev and Marques-Silva 2021), and report the groups in Table 3. Most samples absorb at least one other sample into their group, from 72% on Leaf up to 98–99% on Diabetes and Wine. The mean group size ranges from 2.5 samples on Leaf to 11.9 on Diabetes, and on every dataset but Leaf, 50 to 85% of the

	Diabetes	Glass	Leaf	Parkinsons	Wine
Group Size	11.9	7.3	2.5	6.0	9.8
Build time (s)	52.8	15.3	2.1	76.9	138.6
# Group AXps	1.0	1.1	1.2	8.1	1.3
Solved (%)	100	100	100	99	100
Enum. time (s)	199.1	128.1	19.9	4,023.8	650.3
# AXps	7.3	2.1	2.6	171.4	36.2
Solved (%)	100	100	100	62	100
Enum. time (s)	135.6	22.7	18.7	18,184.4	183.3

Table 3: The top block gives runtime and size statistics for enumerating AXps of the maximal group of a sample (with Algorithm 4) and the bottom block for enumerating the standard AXps of a sample. Solved gives the share of enumerations that finished within the 12-hour time limit, and the runtime are averages over solved instances.

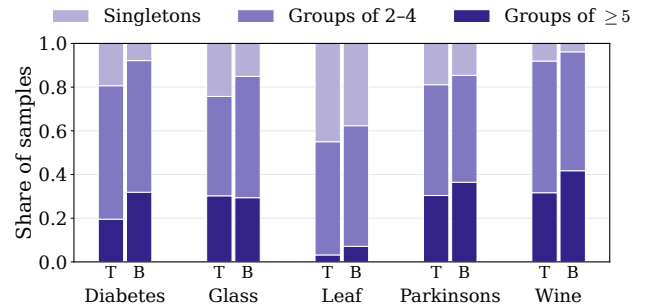


Figure 3: Share of the samples in singletons, in groups of two to four, and in groups of at least five for the top-down (T) and bottom-up (B) cover algorithms.

samples grow a group of at least five. Growing a group takes from 2 seconds on Leaf to 139 on Wine, as the algorithm makes one oracle call per candidate. The oracle times out on a few runs of every dataset except Glass and Leaf.

Table 3 also measures the reduction in the total number of explanations compared to standard AXps. Both enumerations record each explanation as it appears, so a run that hits the twelve-hour timeout reports a lower bound on its count instead of failing. The group cuts the count on every dataset. The samples of Wine average 36 AXps and their groups 1.3, those of Diabetes 7.3 and their groups 1.0, and those of Parkinsons 171 and their groups 8.1. Between 54 and 99% of the groups have a unique group AXp, which is especially desirable, since then the issue of choosing among explanations vanishes. Parkinsons keeps the longest tail, where a handful of samples with small groups still carry up to 121 group AXps.

Finally, Table 3 also shows runtimes for the two enumerations, for samples that did not time out. Enumerating the group takes longer on four datasets, from a near tie on Leaf to five times the sample on Glass, as every oracle call now certifies a box instead of a point. Notably, in Parkinsons, where there are the most explanations, enumerating group AXps is faster. 38% of samples time out when enumerating

	Diabetes	Glass	Leaf	Parkinsons	Wine
Inflated route (AXp, then greedy inflation)					
Solved (%)	52	96	96	3	71
Build time (s)	1,696.1	708.5	465.2	2,255.5	1,487.9
Maximal route (maximal group, then one group AXp)					
Solved (%)	100	100	100	100	100
Build time (s)	97.5	40.2	4.7	283.4	244.8

Table 4: Time to compute one robust explanation via each route over the correctly classified samples of each dataset. Solved gives the share of runs that returned an explanation within the one-hour time limit, and each build time is the average over solved instances.

standard AXps and only 1% when enumerating group AXps. Among non-timeouts, enumerating group AXps averages 67 minutes against 5 hours for the sample, so the drop from 171 AXps to 8 group AXps outweighs the harder calls. Condensing the set of explanations via group AXps can therefore save time as well.

Comparing group AXps and Inflated AXps

Table 4 shows the runtimes for two routes from a sample to one robust explanation. The inflated route computes an AXp and inflates it, and the maximal route grows a maximal group and computes one group AXp of the group. The group AXp approach is faster on every dataset, from 6 times faster on Wine to 99 on Leaf, and it never times out. In contrast, inflating explanations times out on some samples on each dataset, with a 97% failure rate on Parkinsons in particular. The runtime of the group AXp approach scales with the pool of samples, since it makes one oracle call per candidate, so the classes of Leaf, which hold ten samples on average, settle in seconds, while those of Parkinsons, which hold up to 139, take minutes. Inflated AXps and group AXps are of course not the same, but both routes produce a robust explanation. The comparison shows that relaxing the condition of maximally inflating an explanation produces explanations much faster, and ones that still provably cover a set of meaningful samples.

In terms of quality, the maximality of inflated AXps guarantees no width to any individual feature, since maximal bounds can leave a relevant feature i with the zero-width bound $\delta_i^- = \delta_i^+ = x_i$. The algorithm of Izza et al. (2024b) greedily inflates the features it reaches first, leaving the last ones with small bounds. We inflate five AXps of every correctly classified sample with the greedy algorithm of Izza et al. (2024b) at precision $\eta = 0.01$, shuffling the deletion and inflation orders. We find that inflation fixes a feature in 17% of the explanations of Diabetes and 46% of those of Wine, and, remarkably, in at least 95% of those of Glass, Leaf, and Parkinsons. Most general inflated AXps, on the other hand, are the optimal variant of inflated AXps, and maximize the region of covered samples instead of each interval on its own, but computing them is even slower (Izza et al. 2025).

Conclusion

We investigated group AXps, semi-local AXps that hold for every member of a group of samples at once. A group AXp reads its bounds off the spread of its group, so the difficulty moves from growing the bounds to finding the right groups. We gave greedy algorithms that cover a large set of samples and that grow a maximal group around a single sample. On five tabular datasets, a cover describes a class with 1.7 to 3.5 times fewer explanations than one AXp per sample, and a maximal group leaves 54 to 99% of its samples with exactly one group AXp. A group AXp uses $d + 1$ oracle calls, a single call more than a standard AXp, so growing a group and explaining it runs 6 to 99 times faster than greedy inflation, which moreover often pins a feature to a single value. These results suggest that group AXps are a promising semi-local explanation method that can be computed with a relatively small overhead compared to standard AXps, while offering the benefit of more robust explanations that are guaranteed to be valid for a neighbourhood of meaningful samples.

There are various promising directions for future work. The bounding box is axis-aligned, so an outlying member forces the explanation to certify empty space, and our greedy algorithms guarantee neither the fewest groups nor a maximum group. Optimal covers and maximum groups therefore remain open, though both problems are computationally demanding. Extending the group explanation idea to explanations that can take more general shapes, such as space explanations (Labaff et al. 2025), might benefit group explanations in particular, by having the shape more closely follow the values of the samples in the group. Tighter regions would follow the samples more closely, and a cheaper oracle, as tree ensembles offer, would let a group grow over a far larger pool. And the greedy inflation of Izza et al. (2024b) could start from a group’s bounds rather than from a point, where every feature that varies across the group already begins wider than zero.

References

- Aeberhard, S.; and Forina, M. 1992. Wine Dataset.
- Audemard, G.; Bellart, S.; Bounia, L.; Koriche, F.; Lagniez, J.; and Marquis, P. 2022. On the explanatory power of Boolean decision trees. *Data Knowl. Eng.*, 142: 102088.
- Audemard, G.; Lagniez, J.; Marquis, P.; and Szczepanski, N. 2023. Computing Abductive Explanations for Boosted Trees. In Ruiz, F. J. R.; Dy, J. G.; and van de Meent, J., eds., *International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4699–4711. PMLR.
- Bodria, F.; Giannotti, F.; Guidotti, R.; Naretto, F.; Pedreschi, D.; and Rinzivillo, S. 2023. Benchmarking and survey of explanation methods for black box models. *Data Min. Knowl. Discov.*, 37(5): 1719–1778.
- Carrizosa, E.; Ramírez-Ayerbe, J.; and Morales, D. R. 2024a. Generating collective counterfactual explanations in score-based classification via mathematical optimization. *Expert Syst. Appl.*, 238(Part E): 121954.

- Carrizosa, E.; Ramírez-Ayerbe, J.; and Morales, D. R. 2024b. Mathematical optimization modelling for group counterfactual explanations. *Eur. J. Oper. Res.*, 319(2): 399–412.
- Chinneck, J. W.; and Dravnieks, E. W. 1991. Locating Minimal Infeasible Constraint Sets in Linear Programs. *INFORMS J. Comput.*, 3(2): 157–168.
- Confalonieri, R.; Coba, L.; Wagner, B.; and Besold, T. R. 2021. A historical perspective of explainable Artificial Intelligence. *WIREs Data Mining Knowl. Discov.*, 11(1).
- Efron, B.; Hastie, T.; Johnstone, I.; and Tibshirani, R. 2004. Least angle regression. *The Annals of Statistics*, 32(2): 407–499.
- Fokkema, H.; de Heide, R.; and van Erven, T. 2023. Attribution-based Explanations that Provide Recourse Cannot be Robust. *Journal of Machine Learning Research*, 24: 360:1–360:37.
- German, B. 1987. Glass Identification. Retrieved from OpenML, ID 41.
- Guidotti, R. 2024. Counterfactual explanations and how to find them: literature review and benchmarking. *Data Min. Knowl. Discov.*, 38(5): 2770–2824.
- Huang, X.; and Marques-Silva, J. 2023. From Robustness to Explainability and Back Again. *CoRR*, abs/2306.03048.
- Huang, X.; and Marques-Silva, J. 2024. On the failings of Shapley values for explainability. *Int. J. Approx. Reason.*, 171: 109112.
- Ignatiev, A.; and Marques-Silva, J. 2021. SAT-Based Rigorous Explanations for Decision Lists. In Li, C.; and Manyà, F., eds., *Proceedings of Theory and Applications of Satisfiability Testing*, volume 12831 of *Lecture Notes in Computer Science*, 251–269. Springer.
- Ignatiev, A.; Narodytska, N.; and Marques-Silva, J. 2019. Abduction-Based Explanations for Machine Learning Models. In *Proceedings of The Thirty-Third AAAI Conference on Artificial Intelligence*, 1511–1519. AAAI Press.
- Izza, Y.; Huang, X.; Morgado, A.; Planes, J.; Ignatiev, A.; and Marques-Silva, J. 2024a. Distance-Restricted Explanations: Theoretical Underpinnings & Efficient Implementation. In Marquis, P.; Ortiz, M.; and Pagnucco, M., eds., *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning, KR 2024*.
- Izza, Y.; Ignatiev, A.; Rubin, S.; Marques-Silva, J.; and Stuckey, P. J. 2025. Most General Explanations of Tree Ensembles. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 5463–5471. ijcai.org.
- Izza, Y.; Ignatiev, A.; Stuckey, P. J.; and Marques-Silva, J. 2024b. Delivering Inflated Explanations. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence*, 12744–12753. AAAI Press.
- Izza, Y.; and Marques-Silva, J. 2021. On Explaining Random Forests with SAT. In Zhou, Z., ed., *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2584–2591. ijcai.org.
- Jiang, J.; Leofante, F.; Rago, A.; and Toni, F. 2023. Formalising the Robustness of Counterfactual Explanations for Neural Networks. In Williams, B.; Chen, Y.; and Neville, J., eds., *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, 14901–14909. AAAI Press.
- Jiang, J.; Leofante, F.; Rago, A.; and Toni, F. 2024. Robust Counterfactual Explanations in Machine Learning: A Survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*, 8086–8094. ijcai.org.
- Karimi, A.; Barthe, G.; Balle, B.; and Valera, I. 2020. Model-Agnostic Counterfactual Explanations for Consequential Decisions. In Chiappa, S.; and Calandra, R., eds., *The 23rd International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, 895–905. PMLR.
- Karimi, A.; Barthe, G.; Schölkopf, B.; and Valera, I. 2023. A Survey of Algorithmic Recourse: Contrastive Explanations and Consequential Recommendations. *ACM Comput. Surv.*, 55(5): 95:1–95:29.
- Katz, G.; Barrett, C. W.; Dill, D. L.; Julian, K.; and Kochenderfer, M. J. 2017. Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks. In *Computer Aided Verification - 29th International Conference, CAV 2017*, volume 10426 of *Lecture Notes in Computer Science*, 97–117. Springer.
- Kumar, I. E.; Venkatasubramanian, S.; Scheidegger, C.; and Friedler, S. A. 2020. Problems with Shapley-value-based explanations as feature importance measures. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, 5491–5500. PMLR.
- Labbaf, F.; Kolárik, T.; Blicha, M.; Fedyukovich, G.; Wand, M.; and Sharygina, N. 2025. Space Explanations of Neural Network Classification. In *Computer Aided Verification - 37th International Conference, CAV 2025*, volume 15933 of *Lecture Notes in Computer Science*, 287–303. Springer.
- Lakkaraju, H.; Arsov, N.; and Bastani, O. 2020. Robust and Stable Black Box Explanations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, 5628–5638. PMLR.
- Liffiton, M. H.; Previti, A.; Malik, A.; and Marques-Silva, J. 2016. Fast, flexible MUS enumeration. *Constraints*, 21(2): 223–250.
- Little, M. A. 2007. Parkinsons. Retrieved from OpenML, ID 1488.
- Lundberg, S. M.; and Lee, S. 2017. A Unified Approach to Interpreting Model Predictions. In Guyon, I.; von Luxburg, U.; Bengio, S.; Wallach, H. M.; Fergus, R.; Vishwanathan, S. V. N.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems*, 4765–4774.
- Marques-Silva, J. 2024. Logic-Based Explainability: Past, Present and Future. In Margaria, T.; and Steffen, B., eds.,

- Proceedings of the 12th International Symposium on Leveraging Applications of Formal Methods Verification and Validation*, volume 15222 of *Lecture Notes in Computer Science*, 181–204. Springer.
- Marques-Silva, J.; and Huang, X. 2024. Explainability Is Not a Game. *Commun. ACM*, 67(7): 66–75.
- Marques-Silva, J.; and Ignatiev, A. 2022. Delivering Trustworthy AI through Formal XAI. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence*, 12342–12350. AAAI Press.
- Minh, D.; Wang, H. X.; Li, Y. F.; and Nguyen, T. N. 2022. Explainable artificial intelligence: a comprehensive review. *Artif. Intell. Rev.*, 55(5): 3503–3568.
- Parmentier, A.; and Vidal, T. 2021. Optimal Counterfactual Explanations in Tree Ensembles. In Meila, M.; and Zhang, T., eds., *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, 8422–8431. PMLR.
- Ribeiro, M. T.; Singh, S.; and Guestrin, C. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In Krishnapuram, B.; Shah, M.; Smola, A. J.; Aggarwal, C. C.; Shen, D.; and Rastogi, R., eds., *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, 1135–1144. ACM.
- Ribeiro, M. T.; Singh, S.; and Guestrin, C. 2018. Anchors: High-Precision Model-Agnostic Explanations. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 1527–1535. AAAI Press.
- Shih, A.; Choi, A.; and Darwiche, A. 2018. A Symbolic Approach to Explaining Bayesian Network Classifiers. In Lang, J., ed., *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 5103–5111. ijcai.org.
- Silva, P. F. B.; and Marçal, A. R. S. 2014. Leaf. Retrieved from OpenML, ID 1482.
- Wachter, S.; Mittelstadt, B. D.; and Russell, C. 2018. Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR. *Harvard Journal of Law & Technology*, 31(2): 841–887.
- Wäldchen, S.; MacDonald, J.; Hauch, S.; and Kutyniok, G. 2021. The Computational Complexity of Understanding Binary Classifier Decisions. *J. Artif. Intell. Res.*, 70: 351–387.
- Wang, S.; Zhang, H.; Xu, K.; Lin, X.; Jana, S.; Hsieh, C.; and Kolter, J. Z. 2021. Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Neural Network Robustness Verification. In *Advances in Neural Information Processing Systems 34*, 29909–29921.
- Warren, G.; Keane, M. T.; Guéret, C.; and Delaney, E. 2023. Explaining Groups of Instances Counterfactually for XAI: A Use Case, Algorithm and User Study for Group-Counterfactuals. *CoRR*, abs/2303.09297.
- Wielopolski, P.; Furman, O.; Stefanowski, J.; and Zieba, M. 2024. Unifying Perspectives: Plausible Counterfactual Explanations on Global, Group-wise, and Local Levels. *CoRR*, abs/2405.17642.
- Wu, M.; Wu, H.; and Barrett, C. W. 2023. VeriX: Towards Verified Explainability of Deep Neural Networks. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*.
- Yu, J.; Farr, G.; Ignatiev, A.; and Stuckey, P. J. 2024. Anytime Approximate Formal Feature Attribution. In Chakraborty, S.; and Jiang, J. R., eds., *27th International Conference on Theory and Applications of Satisfiability Testing, SAT 2024, Pune, India, August 21-24, 2024*, volume 305 of *LIPIcs*, 30:1–30:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.